STAGE 1

Setting paths:

```
cd /home/user01/solr-8.11.0/solr-8110-maven-test

R="../results"

mkdir -p "$R"
```

Maven project metadata input:

```
mvn org.apache.maven.plugins:maven-dependency-plugin:3.10.0:tree \
 -DoutputFile="$R/section1_maven_dependency_tree.txt"
```

Parsing log4j:

```
grep "org.apache.logging.log4j:log4j-core" \
 "$R/section1_maven_dependency_tree.txt" \
 | tee "$R/stage1-solr-maven-log4j-core.txt"
```

Syft SBOM tool:

```
syft dir:. \
 -o cyclonedx-json="$R/sbom_syft_manifest.cdx.json"
```

Trivy as SBOM generator:

```
trivy fs \
 --format cyclonedx \
 --output "$R/sbom_trivy_manifest.cdx.json" \
 .
```

Cdxgen SBOM tool:

```
cdxgen -t maven \
 -o "$R/sbom_cdxgen_manifest.cdx.json" \
 .
```

Maven resolved JAR directory input :

```
mvn dependency:copy-dependencies \
 -DincludeScope=runtime \
 -DoutputDirectory=target/dependency
```

Syft SBOM tool:

```
syft dir:target/dependency \
 -o cyclonedx-json="$R/sbom_syft_resolved_jars.cdx.json"
```

Trivy SBOM tool:

```
trivy fs \
 --format cyclonedx \
 --output "$R/sbom_trivy_resolved_jars.cdx.json" \
 target/dependency
```

Cdxgen SBOM tool:

```
cdxgen -t java \
 -o "$R/sbom_cdxgen_resolved_jars.cdx.json" \
 target/dependency
```

Extract log4j-core from all SBOMs:

```
for f in "$R"/sbom_*.cdx.json; do
 echo "==== $f ===="
 jq -r '
  .components[]?
  | select(
     (.name // "" | test("log4j-core"; "i"))
     or
```

```
      (.purl // "" | test("log4j-core"; "i"))
    )
  | [.group, .name, .version, .type, .purl]
  | @tsv
' "$f"
done | tee "$R/stage1-solr-sbom-log4j-core-results.tsv"
```

Grype vulnerability scan on SBOMs:

```
grype sbom:"$R/sbom_syft_manifest.cdx.json" \
 --by-cve -o json > "$R/grype_sbom_syft_manifest.json"

grype sbom:"$R/sbom_syft_resolved_jars.cdx.json" \
 --by-cve -o json > "$R/grype_sbom_syft_resolved_jars.json"

grype sbom:"$R/sbom_trivy_manifest.cdx.json" \
 --by-cve -o json > "$R/grype_sbom_trivy_manifest.json"

for f in "$R"/grype_*.json; do
 echo "==== $f ===="
 jq -r '
  .matches[]?
  | select(
     (.artifact.name // "" | test("log4j-core"; "i"))
     or
     (.artifact.purl // "" | test("log4j-core"; "i"))
    )
```

```
  | [
      .artifact.name,
      .artifact.version,
      .artifact.type,
      .artifact.purl,
      .vulnerability.id,
      .vulnerability.severity,
      (.matchDetails[0].type // "n/a"),
      (.matchDetails[0].matcher // "n/a")
    ]
  | @tsv
' "$f"
done | tee "$R/stage1-solr-grype-log4j-results.tsv"
```

Trivy vulnerability scan on SBOMs:

```
trivy sbom "$R/sbom_syft_manifest.cdx.json" \
  --format json \
  --output "$R/trivy_sbom_syft_manifest_vuln.json"

trivy sbom "$R/sbom_syft_resolved_jars.cdx.json" \
  --format json \
  --output "$R/trivy_sbom_syft_resolved_jars_vuln.json"

trivy sbom "$R/sbom_trivy_manifest.cdx.json" \
  --format json \
  --output "$R/trivy_sbom_trivy_manifest_vuln.json"
```

```
for f in "$R"/trivy_*_vuln.json; do
  echo "==== $f ===="
  jq -r '
    .Results[]?.Vulnerabilities[]?
    | select(
        (.PkgName // "" | test("log4j-core"; "i"))
        or
        (.PkgIdentifier.PURL // "" | test("log4j-core"; "i"))
      )
    | [
        .PkgName,
        .InstalledVersion,
        .VulnerabilityID,
        .Severity,
        (.FixedVersion // "")
      ]
    | @tsv
  ' "$f"
done | tee "$R/stage1-solr-trivy-log4j-vulns.tsv"
```